

Sistemas Distribuidos I (75.74)

Coffee Shop Analysis

Introducción

El presente trabajo práctico, titulado *Coffee Shop Analysis*, se desarrolla en el marco de la asignatura **Sistemas Distribuidos I (75.74)**, con el objetivo de diseñar e implementar un sistema distribuido orientado al análisis de información transaccional de una cadena de cafeterías en Malasia.

La propuesta surge de la necesidad de procesar grandes volúmenes de datos relacionados con las ventas, los clientes, las tiendas y los productos ofrecidos por la empresa. Dicho procesamiento debe permitir obtener métricas de negocio clave tales como:

- Identificación de transacciones relevantes,
- Ranking de productos más vendidos y más rentables,
- Cálculo del *Total Payment Value* (TPV) por sucursal en distintos períodos,
- Detección de clientes más frecuentes junto con información asociada.

El diseño de la solución requiere considerar **requerimientos funcionales y no funcionales propios de un sistema distribuido**. En particular, se busca garantizar escalabilidad en entornos multi-computadora, soportar la abstracción de comunicación mediante un *middleware* desarrollado sobre interfaces provistas por la cátedra, y asegurar robustez frente a la finalización del sistema mediante un *graceful quit*.

Análisis

Contexto

La cadena de cafeterías a analizar genera diariamente una gran cantidad de transacciones. La información a procesar no solo incluye los datos de cada venta, sino también la relación con los productos, las sucursales y los clientes, lo que demanda una solución capaz de coordinar distintos procesos en paralelo y soportar la escalabilidad del sistema a medida que el volumen de datos crece.

1. Vista Lógica

Actores

- **Cliente (Cafetería en Malasia):** punto de entrada, envía los archivos de ventas (`transactions_YYYYMM.csv`, `transaction_items_YYYYMM.csv`, `users_YYYYMM.csv, menu_items.csv`) al sistema.

- **Sistema distribuido:** recibe, procesa y categoriza la información mediante filtros y categorizadores, produciendo resultados para cada requerimiento.

- **Analista/Administrador:** consulta los resultados (reportes y base de resultados).

Funcionalidad

El sistema permite:

1. **Filtrado de transacciones**: obtener aquellas realizadas en 2024 y 2025, dentro de la franja horaria de 06:00 a 23:00, con montos mayores o iguales a 75.

2. **Ranking de productos**: identificar los más vendidos y los que mayor ganancia generan, discriminados por mes.

3. **Cálculo del TPV**: determinar el *Total Payment Value* por semestre y por sucursal, respetando las restricciones horarias.

4. **Clientes frecuentes**: obtener la fecha de cumpleaños de los tres clientes con mayor cantidad de compras en cada sucursal.

Controladores y Flujo General

El procesamiento está dividido en **contenedores independientes** que se comunican de manera asíncrona a través de **RabbitMQ**. Cada filtro o categorizador corre en contenedores Docker **replicados** para asegurar resiliencia y tolerancia a fallos.

Flujo detallado:

1. **Ingreso de datos:**

    - El *Cliente (Cafetería)* envía los archivos de entrada al *Boundary/Gateway*.

    - El *Gateway* publica los archivos en **RabbitMQ**.

2. **Pre-filtrado temporal (Filter 2024/2025 – created_at):**

- Filtra las filas por el campo `created_at` para quedarse únicamente con transacciones de 2024 y 2025.

- Los datos de `transactions_items_YYYYMM.csv` se envían al **Categorizer de Query 2**.

- Los datos de `transactions_YYYYMM.csv` se envían tanto a:

  - **Filter 6AM–11PM** (para Query 1 y Query 3).

  - **Categorizer de Query 4** (para identificar clientes más frecuentes por sucursal).

3. **Procesamiento por queries:**

   - **Query 1 (Transacciones >= 75):**
     `transactions_YYYYMM.csv` → *Filter 6AM–11PM* → *Filter Total ≥ 75* → resultados con ID y monto de las transacciones válidas.

   - **Query 2 (Productos más vendidos / más ganancias):**
     `transaction_items_YYYYMM.csv` → *Categorizer Query 2* → resultados mensuales de 2024–2025 -> *Product Name Mapper Query 2* (requiere join con `menu_items.csv`).

   - **Query 3 (TPV por semestre/sucursal):**
     `transactions_YYYYMM.csv` → *Filter 6AM–11PM* → *Categorizer Sucursal Semestre* → resultados semestrales por sucursal.

   - **Query 4 (Top 3 clientes por sucursal):**
     `transactions_YYYYMM.csv` → *Categorizer Query 4* → resultados con top 3 clientes -> *Client Birthday Dictionary Query 4* -> resultados con fechas de cumpleaños (requiere join con `users_YYYYMM.csv`).

4. **Middleware distribuido:**

   - Todos los microservicios se comunican por **RabbitMQ**.

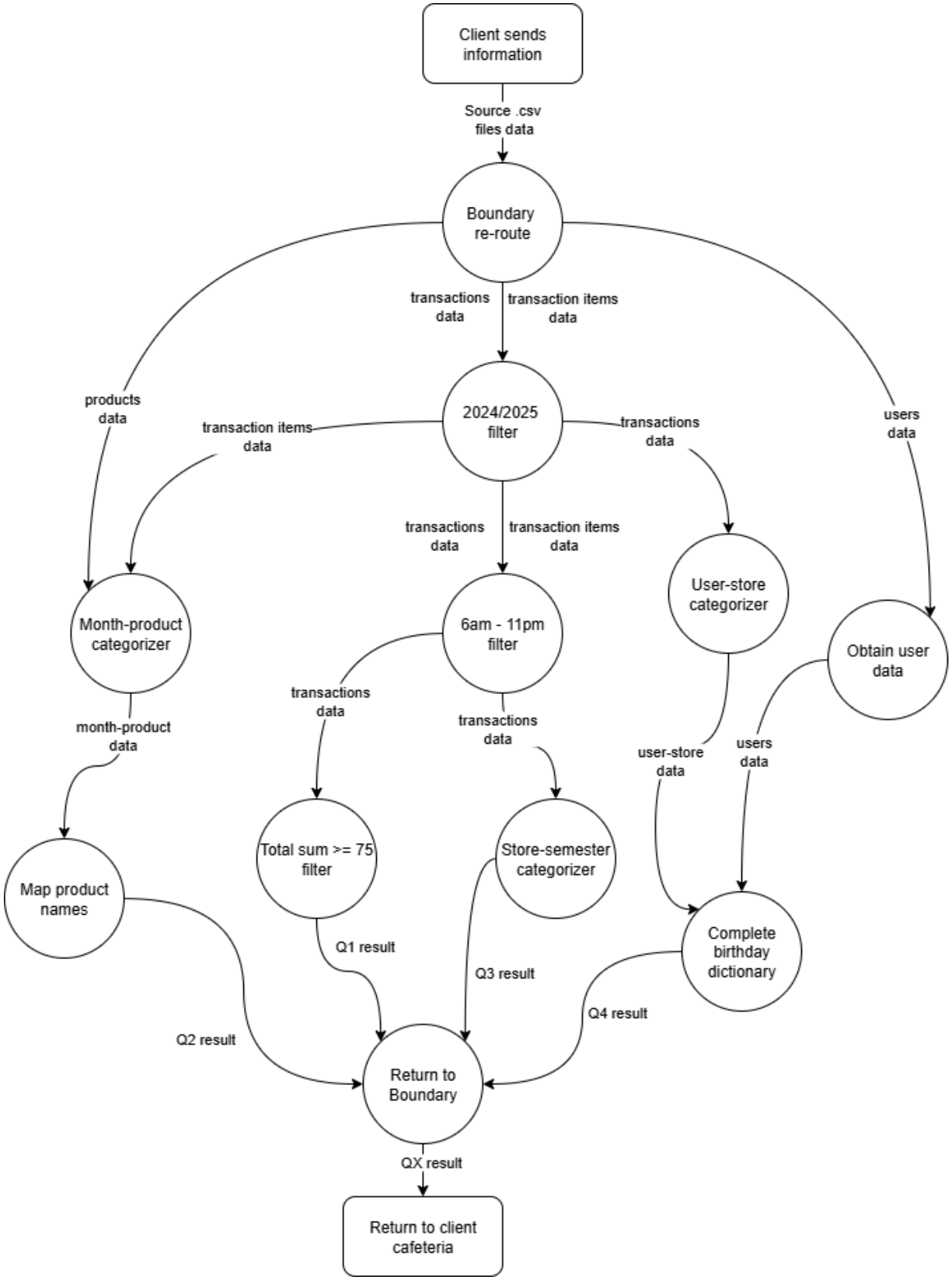
   - Los filtros y categorizadores están desplegados en **contenedores Docker independientes y replicados**, garantizando resiliencia.

5. **Resiliencia y tolerancia a fallos:**

   - El sistema soporta **escalamiento horizontal** (se pueden agregar más instancias de filtros/categorizadores).

   - Cada servicio maneja un **graceful quit** en caso de recibir señal `SIGTERM`.

Diagrama DAG



2. Vista de Procesos

La vista de procesos describe la **dinámica de ejecución** del sistema, con foco en concurrencia, comunicación y tolerancia a fallos.

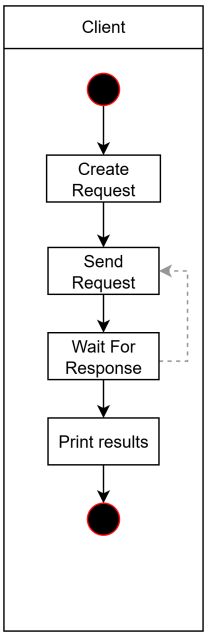
Procesamiento distribuido

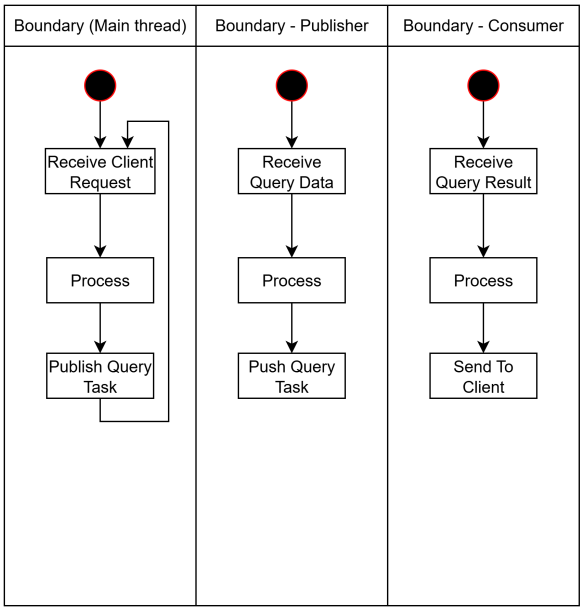
- Cada filtro y categorizador es un contenedor Docker independiente.
- Los contenedores están duplicados para brindar **resiliencia** ante fallos (si uno cae, otro sigue procesando).
- Se emplea **RabbitMQ** como middleware para comunicación asíncrona y desacoplada entre componentes.

Flujos principales

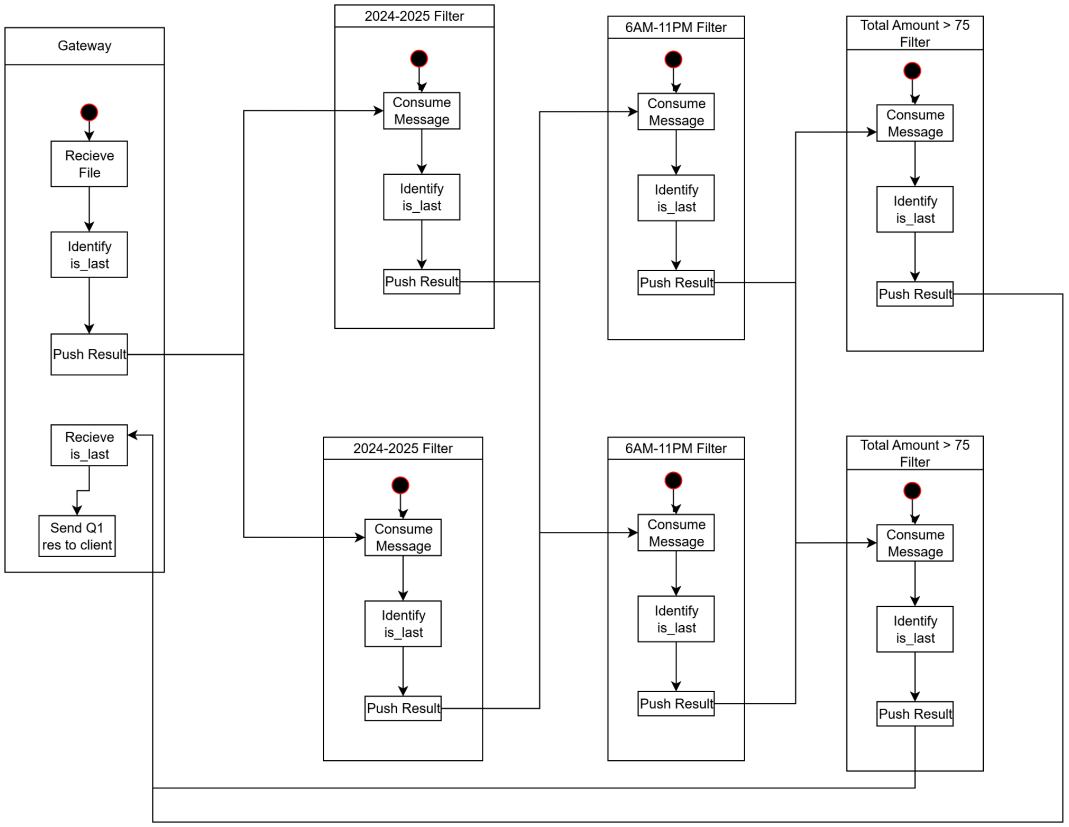
- El cliente sube los archivos al Boundary→el Boundary los envía en batches a RabbitMQ.
- Los filtros procesan los datos en paralelo (por año, por horario, por monto).
- Los categorizadores reciben datos filtrados y los procesan para queries específicas.
- El boundary lee la salida de la queue correspondiente a cada query genera la salida final.

Diagramas de actividades

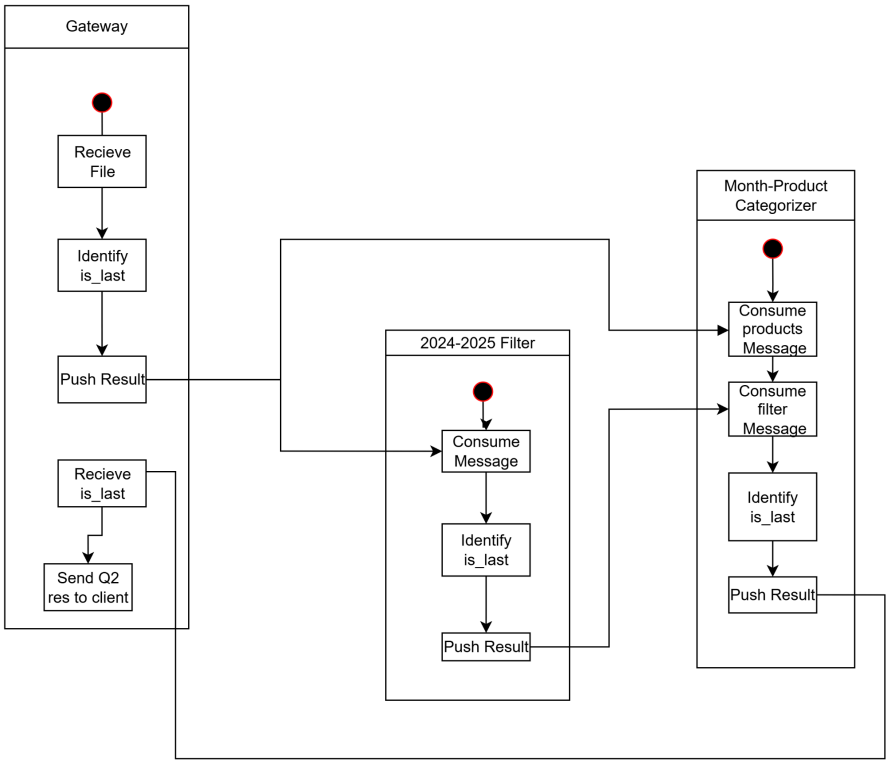




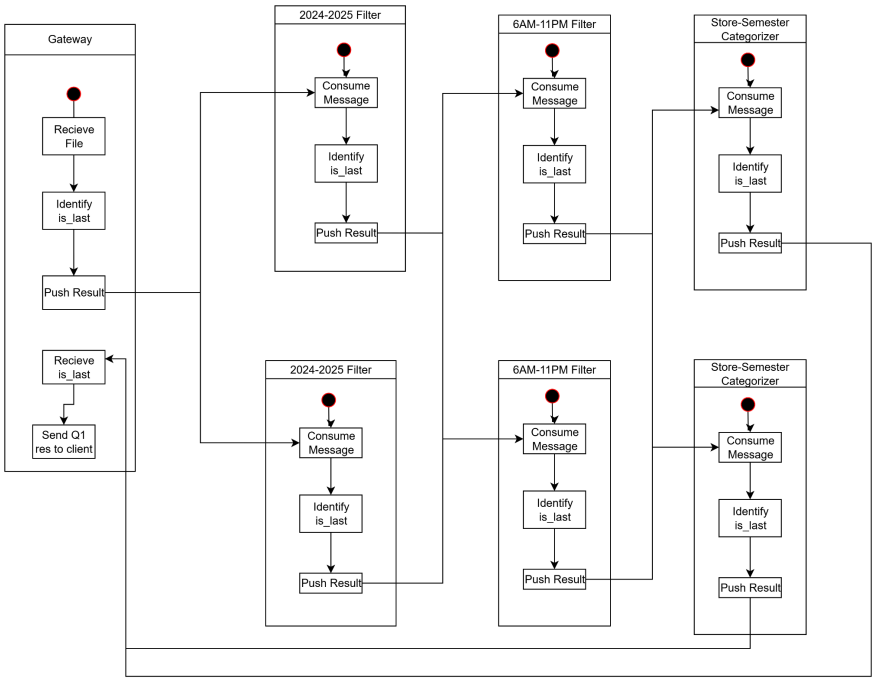
Procesamiento Q1



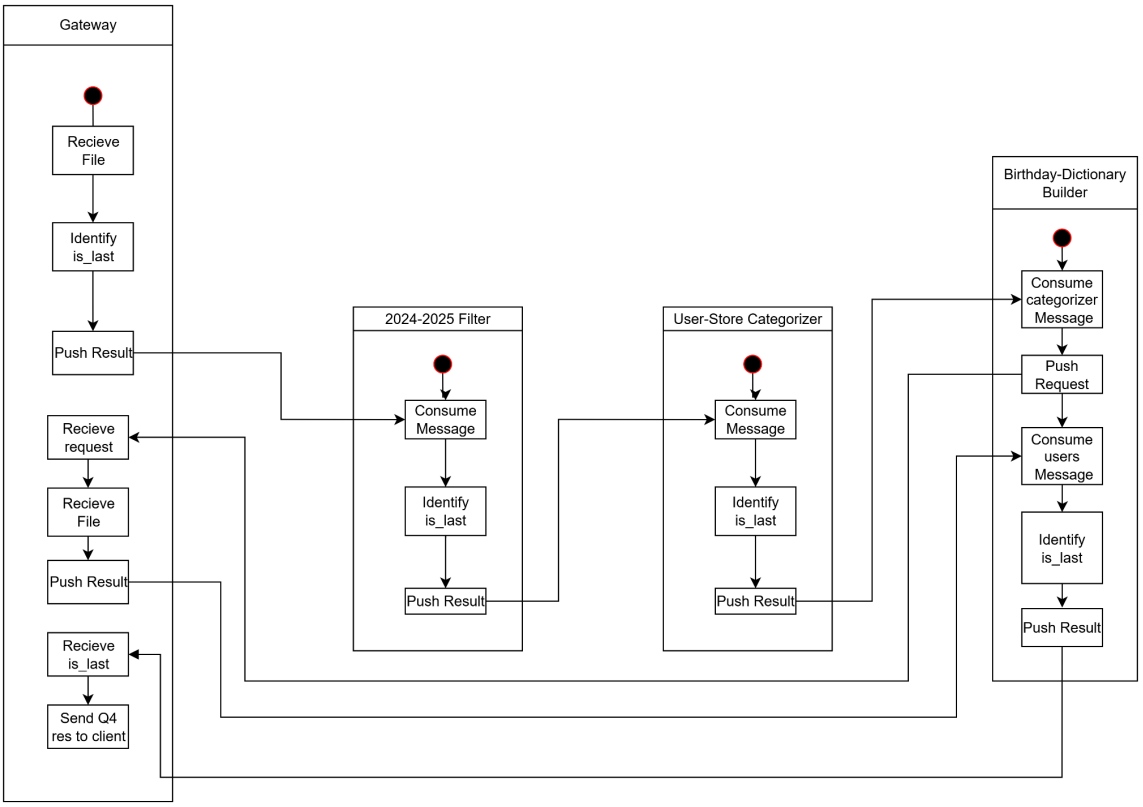
Procesamiento Q2



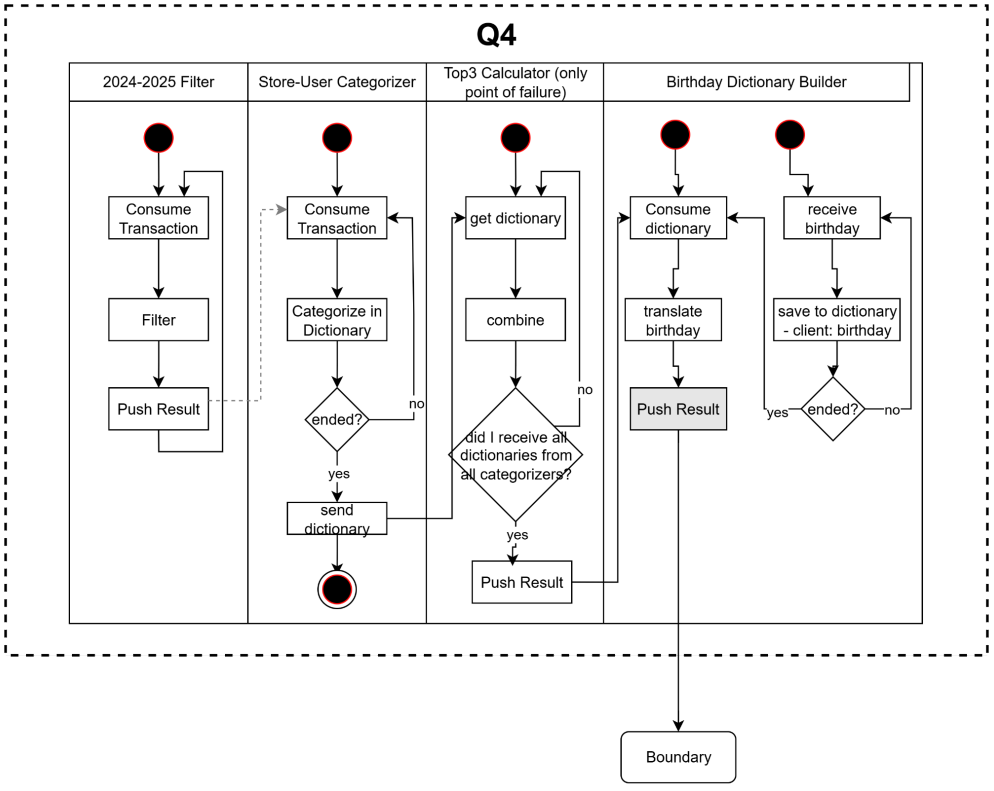
Procesamiento Q3



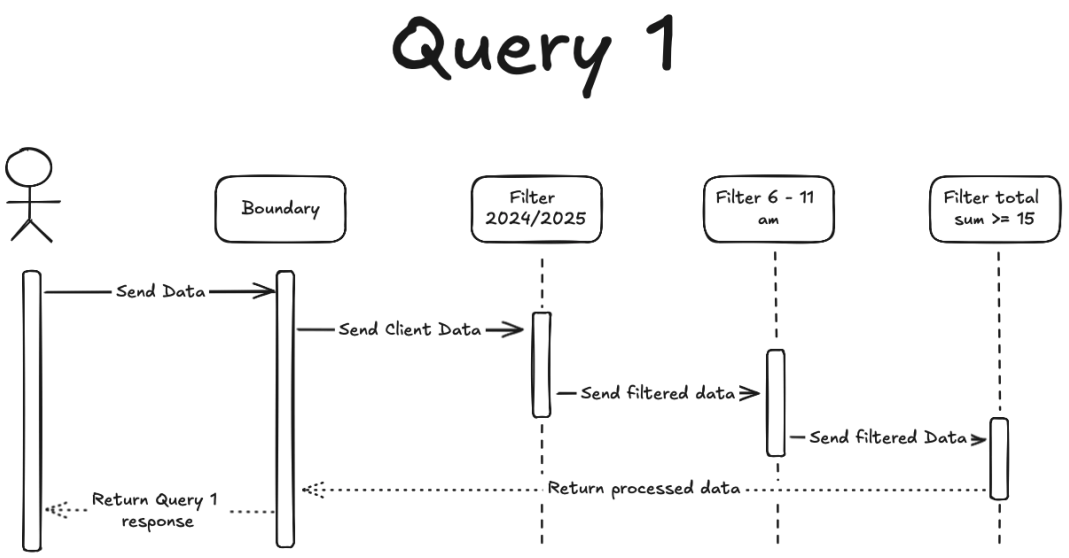
Procesamiento Q4

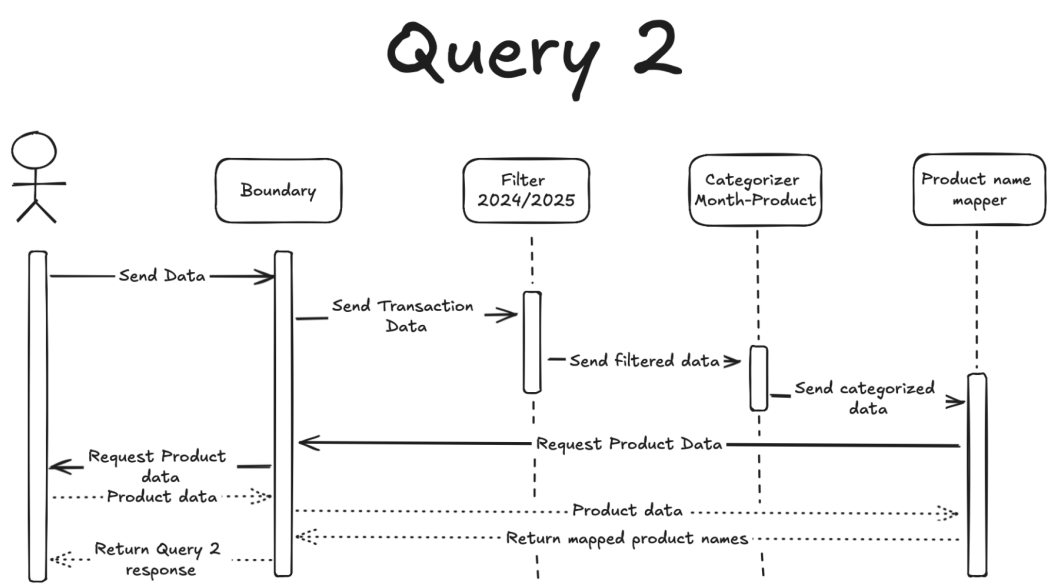


Zoom

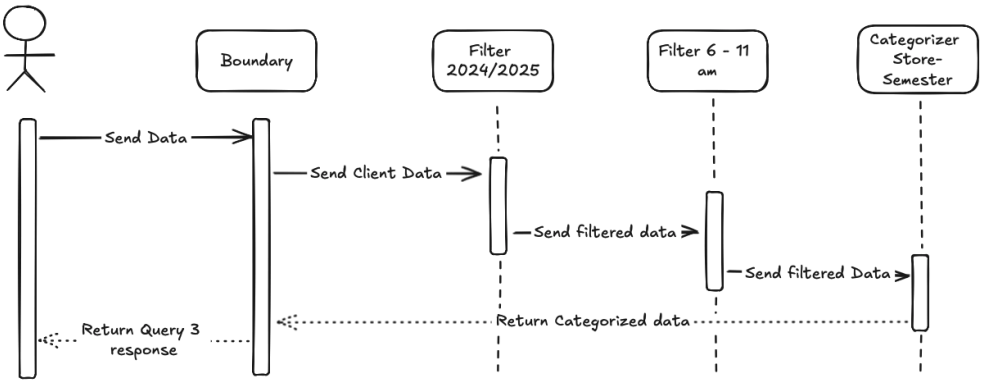


Diagramas de secuencia

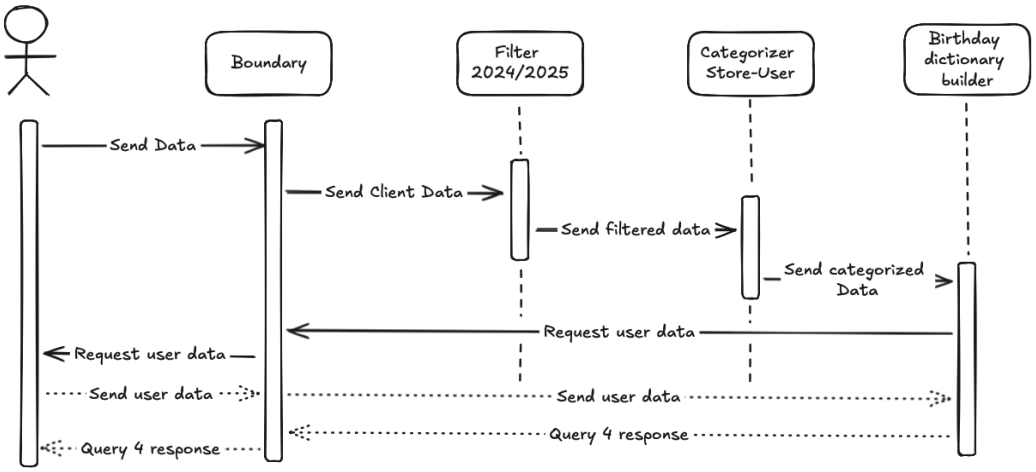




Query 3



Query 4



---

3. Vista de Desarrollo

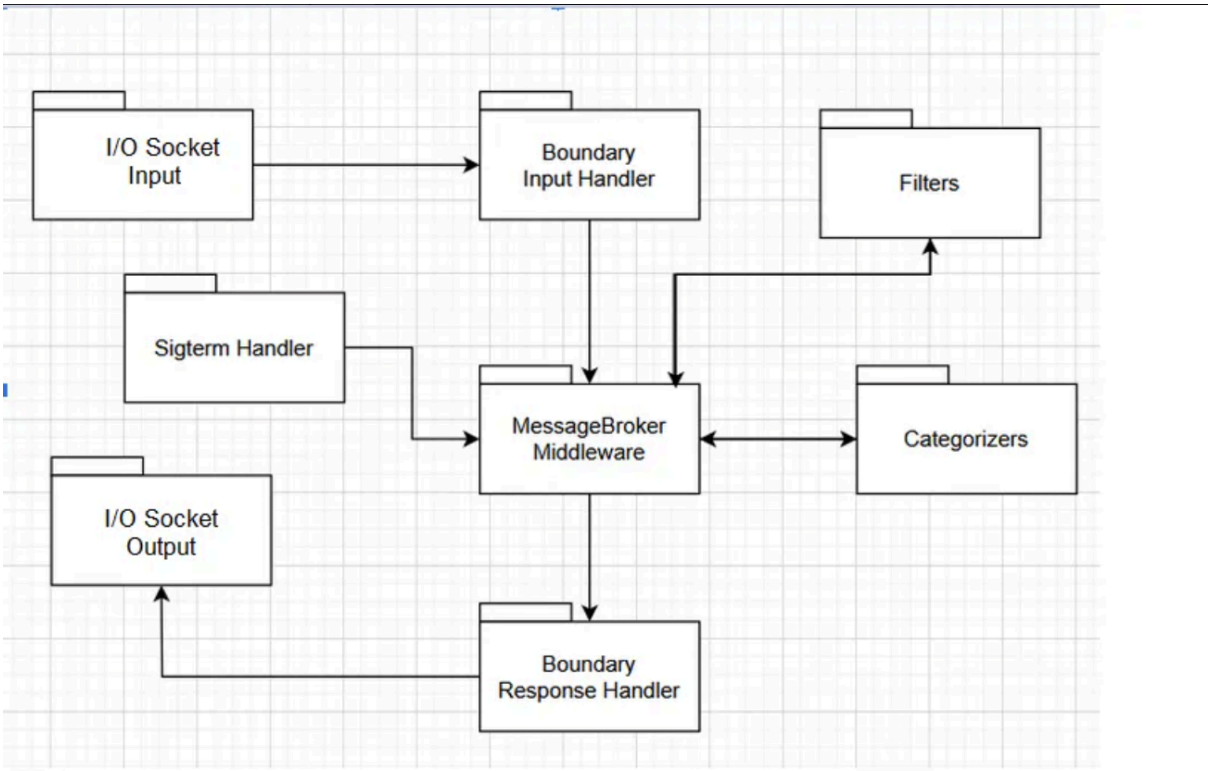
Organización en paquetes

- **Input:** Cliente y Gateway.
- **Messaging:** RabbitMQ como middleware.
- **Filters:** módulos reutilizables (FilterYear, FilterTime, FilterAmount).
- **Categorizers:** módulos de categorización por productos, TPV y clientes.
- **Sigterms:** módulo dentro del Boundary que maneja Sigterms recibidas del client y las distribuye por todo el sistema.

Esta organización promueve:

- **Reutilización** (los filtros son comunes a varias queries).
- **Separación de responsabilidades** (cada paquete tiene un rol claro).

Diagrama de Paquetes



---

4. Vista Física

La vista física describe cómo se despliega el sistema en la infraestructura.

Componentes de infraestructura

- **Cliente (Cafetería):** máquina que envía los archivos.
- **Boundary:** recibe conexiones, valida datos y los distribuye por RabbitMQ, devuelve resultados, distribuye sigterms.
- **RabbitMQ:** garantiza mensajería distribuida y tolerante a fallos.
- **Contenedores Docker:** cada filtro, categorizador y query handler corre en su propio contenedor (posibilidad de escalar horizontalmente, y tolerancia a fallos).

Características

- **Escalabilidad:** al aumentar volumen de datos, se pueden levantar más instancias de filtros o categorizadores.
- **Resiliencia:** redundancia de contenedores y tolerancia a señales `SIGTERM`.
- **Middleware abstracto:** RabbitMQ asegura comunicación desacoplada entre módulos.

Diagrama de Robustez

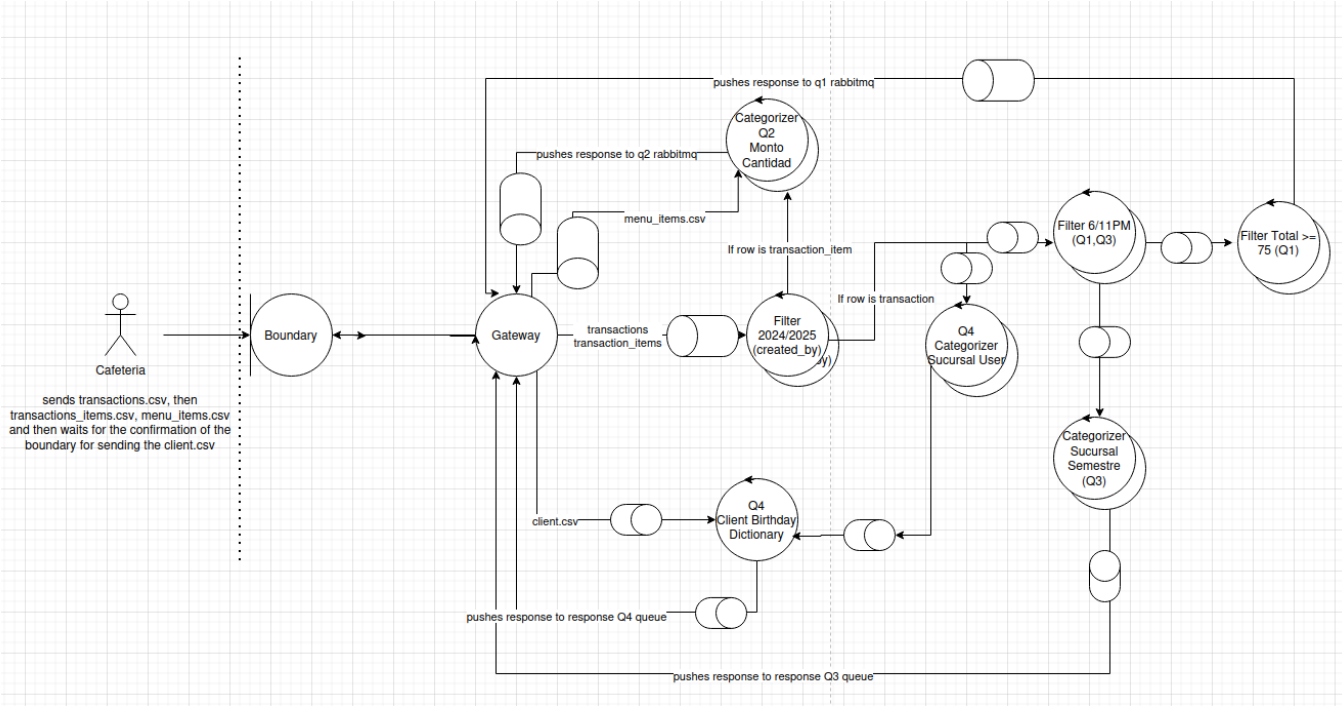
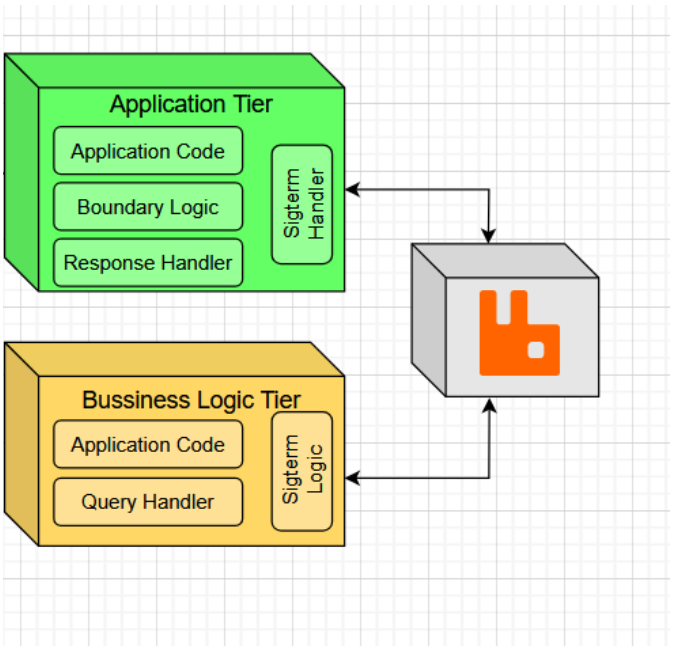


Diagrama Despliegue



---

5. Scenarios (+1)

Finalmente, se presentan escenarios que integran las vistas anteriores.

Escenario 1: Query1 (Transacciones ≥75)

1. Cliente envía `transactions_YYYYMM.csv`.
2. Boundary lo transmite a RabbitMQ.
3. Filtros aplican: `FilterYear` → `FilterTime` → `FilterAmount`.
4. Resultado devuelto por el Boundary.

Escenario 2: Query2

1. Cliente envía `transaction_items_YYYYMM.csv`.
2. Boundary lo transmite a RabbitMQ.
3. Filtros aplican: `FilterYear`.
4. Se utiliza el `Month-Product Categorizer`.
5. Se envía señal mediante RabbitMQ y los resultados se envían a `Product-Name Mapper`.
6. El Boundary pide al cliente los datos de productos.
7. Cliente envía `menu_items.csv`.
8. El Boundary los transmite a RabbitMQ.
9. `Product-Name Mapper` realiza el mapeo de nombres con los resultados categorizados.
10. Resultado devuelto por el Boundary.

Escenario 3: Query3

1. Cliente envía `transaction_items_YYYYMM.csv`.
2. Boundary lo transmite a RabbitMQ.
3. Filtros aplican: `FilterYear, FilterHour`.
4. Se utiliza el `CategorizerStoreSemester`
5. Resultado devuelto por el Boundary.

Escenario 4: Query4

1. Cliente envía `transactions_YYYYMM.csv`.
2. Gateway envía ambos a RabbitMQ.
3. `FilterYear` procesa.
4. `CategorizerClients` realiza la categorización de clientes.
5. Se envía señal mediante RabbitMQ y los resultados se envían a `Client Birthday Mapper`.
6. El Boundary pide al cliente los datos de clientes.
7. Cliente envía los archivos `users_YYYYMM.csv`.
8. El Boundary los transmite a RabbitMQ del `Client Birthday Mapper`.
9. El `Client Birthday Mapper` selecciona top 3 clientes y obtiene cumpleaños.

Escenario Sigterm:

1. El Boundary recibe un Sigterm.
2. El Boundary publica el Sigterm en el Topic de Sigterms de RabbitMQ, al que todos los otros nodos están suscritos.
3. Al leer ese mensaje todos los nodos hacen un shutdown gracefully.

Tolerancia a fallas:

Categorizers:

Manejo de Datos en Caso de Falla y Reactivación del Nodo para los categorizers:

El código de los categorizers implementa un mecanismo robusto para manejar fallas y garantizar la persistencia de datos. Este mecanismo asegura que, en caso de que el nodo falle y se reactive, el estado previo se recupere correctamente y los mensajes no se procesen dos veces. A continuación, se detalla cómo se gestiona la persistencia, recuperación y procesamiento de datos.

1. Persistencia de Datos

El nodo utiliza dos archivos principales para la persistencia de datos:

BACKUP FILE: Archivo principal donde se guarda el estado persistente del nodo.

AUXILIARY FILE: Archivo temporal utilizado para escribir el estado de manera atómica antes de renombrarlo como BACKUP FILE.

Los archivos de respaldo guardan:

- La data acumulada hasta ahora como corresponde a cada categorizer.
- Los mensajes finales de cada cliente (y su cantidad).
- El último mensaje de cada sender
- Los últimos n mensajes que no se llegaron a escribir al diccionario persistido. Los mensajes están codificados en Base64.

Guardado del Estado

El estado se guarda periódicamente en el archivo de respaldo:

- Serializar el estado actual del nodo en formato JSON.
- Escribir el estado en el archivo temporal (AUXILIARY FILE).
- Renombrar el archivo temporal como BACKUP FILE de manera atómica.

El renombre del archivo es importante porque utiliza una syscall atómica. Esto permite identificar, sin posibilidad de corte intermedio, si el archivo auxiliar llegó a actualizar el archivo de backup o no.

2. Recuperación de Datos

En caso de que el nodo falle y se reactive, el estado se recupera desde el archivo de respaldo. Este proceso incluye:

Descartar el archivo temporal (AUXILIARY FILE):

Si el archivo temporal existe, se elimina para evitar inconsistencias.

Leer el archivo de respaldo (BACKUP FILE):

La primera línea (JSON) se utiliza para restaurar el estado del nodo.

Las líneas restantes (mensajes en Base64) se decodifican y se almacenan para su reprocesamiento.

3. Reprocesamiento de Mensajes

Los mensajes recuperados se reprocesan a partir del archivo de respaldo. Esto asegura que los mensajes que no se completaron antes de la falla sean procesados nuevamente.

Manejo de Mensajes Duplicados

Para evitar procesar mensajes duplicados, el nodo utiliza el diccionario contenido dentro del json que almacena el último mensaje procesado por cada remitente. Si un mensaje ya ha sido procesado, se ignora.

Procesamiento de Mensajes Recuperados

Los mensajes recuperados se procesan de la misma manera que los mensajes nuevos:

Se verifican los campos requeridos.

Se actualizan los datos de mensajes finales y el diccionario.

Filters:

Cada filter worker utiliza un archivo de persistencia por remitente (sender) para guardar el último message_id procesado. Estos archivos se almacenan en su directorio designado.

Archivos para persistir:

El archivo de persistencia para un remitente específico se genera dinámicamente.

Contenido del archivo:

Cada archivo contiene únicamente el último message_id procesado para el remitente correspondiente.

Guardado del Último Mensaje Procesado

Después de procesar un mensaje exitosamente, el worker guarda el message_id en el archivo de persistencia.

2. Recuperación de Datos

Cuando el worker se reinicia, carga el último message_id procesado para cada remitente desde los archivos de persistencia.

3. Manejo de Mensajes Duplicados

Para evitar procesar mensajes duplicados después de una caída y reconexión, el worker compara el message_id del mensaje recibido con el último message_id guardado en el archivo de persistencia.

Birthday_dictionary

Archivos de Respaldo

El nodo utiliza dos archivos principales para la persistencia de datos:

BACKUP_FILE_TOP_USERS: Guarda el estado relacionado con los usuarios principales (top_users) y los mensajes procesados.

BACKUP_FILE_USERS_DATA: Guarda los datos de cumpleaños (birthdays_per_client) y el último mensaje procesado globalmente (g_last_message_id).

Cada archivo tiene un archivo auxiliar temporal (**AUXILIARY_FILE_TOP_USERS** y **AUXILIARY_FILE_USERS_DATA**) que se utiliza para escribir los datos de manera atómica antes de renombrarlo como el archivo principal.

Estructura de los Archivos de Respaldo

BACKUP_FILE_TOP_USERS:

messages_by_client: Mensajes acumulados por cliente.

message_count_by_client: Contador de mensajes recibidos por cliente.

last_message_per_sender: Último mensaje procesado por cada remitente.

BACKUP_FILE_USERS_DATA:

passed_top_users: Usuarios principales procesados por cliente.

birthdays_per_client: Fechas de cumpleaños por cliente.

g_last_message_id: Último mensaje procesado globalmente.

Guardado del Estado

El estado se guarda periódicamente en los archivos de respaldo mediante las funciones save_top_users_state_to_disk y save_users_data_state_to_disk. Estas funciones:

Serializan el estado actual en formato JSON.

Escriben los datos en el archivo auxiliar.

Renombran el archivo auxiliar como el archivo principal de manera atómica.

Recuperación de datos

Cuando el nodo se reinicia, recupera el estado desde los archivos de respaldo mediante las funciones recover_top_users_state y recover_users_data_state.

Flujo de Recuperación:

Descartar Archivos Auxiliares:

Si existen archivos auxiliares (**AUXILIARY_FILE_TOP_USERS** o **AUXILIARY_FILE_USERS_DATA**), se eliminan para evitar inconsistencias.

Leer Archivos de Respaldo:

Si los archivos de respaldo existen, se leen y se restauran los datos.

Si no existen, el nodo comienza desde cero.

Restaurar Estado:

Se restauran los datos de usuarios principales (messages_by_client, message_count_by_client) y los datos de cumpleaños (birthdays_per_client).

Reiniciar Hilos de Clientes:

Para cada cliente en passed_top_users, se reinicia un hilo (BirthClientHandler) para continuar el procesamiento.

Manejo de Mensajes Duplicados

Para evitar procesar mensajes duplicados después de una caída y reconexión, el nodo utiliza el diccionario last_message_per_sender, que almacena el último message_id procesado por cada remitente.

Lógica de Detección de Duplicados:

Cuando se recibe un mensaje, se verifica si su message_id coincide con el último message_id procesado para el remitente.

Si el mensaje es duplicado, se ignora y se envía un ACK para evitar su reentrega.

Procesamiento de Mensajes

Si el mensaje no es duplicado, el nodo lo procesa normalmente:

Acumular Usuarios Principales:

Los datos de usuarios principales (top_users) se acumulan en messages_by_client.

Procesar Datos de Cumpleaños:

Los datos de cumpleaños se procesan en hilos separados (BirthClientHandler) para cada cliente.

Guardar Estado:

Después de procesar el mensaje, el estado se guarda en los archivos de respaldo.

Enviar ACK:

Finalmente, se envía un ACK al sistema de mensajería para confirmar que el mensaje fue procesado correctamente.

Healthcheckers:

Función y Funcionamiento de los Health Checkers

El nodo health_checker tiene como objetivo monitorear la salud de otros nodos en el sistema distribuido. Esto se logra mediante el envío periódico de mensajes de "alive" (salud) a los nodos supervisados. Si un nodo supervisado no responde, el health_checker reinicia el contenedor Docker correspondiente para intentar restaurar su funcionamiento.

Función de los Health Checkers

Propósito Principal

Monitorear la Disponibilidad: Verificar que los nodos supervisados estén activos y funcionando correctamente.

Reiniciar Nodos Caídos: Si un nodo supervisado no responde, el health_checker detiene y reinicia el contenedor Docker asociado al nodo.

Objetivo General

Garantizar la resiliencia del sistema distribuido al detectar y recuperar automáticamente nodos que hayan fallado.

Flujo de Ejecución

Inicio del health_checker:

El nodo health_checker se inicializa y crea un hilo (HealthCheckerHandler) para cada nodo supervisado.

Las direcciones de los nodos supervisados se especifican en la variable de entorno HEALTH_ADDRESS_TARGETS.

Conexión con el Nodo Supervisado:

Cada hilo intenta conectarse al nodo supervisado mediante un socket TCP.

Si la conexión es exitosa, comienza a enviar mensajes de "alive".

Envío de Mensajes de "Alive":

Cada segundo, el health_checker envía un entero (1) al nodo supervisado utilizando el método send_int.

El nodo supervisado debe responder con un entero (1) utilizando el método recv_int.

Detección de Fallas:

Si el health_checker no puede establecer una conexión o no recibe una respuesta del nodo supervisado, asume que el nodo ha fallado.

Reinicio del Nodo Caído:

El health_checker ejecuta los siguientes comandos de Docker:

docker stop: Detiene el contenedor del nodo caído.

docker start: Reinicia el contenedor del nodo caído.

Después de reiniciar el contenedor, el health_checker intenta reconectarse al nodo.